TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN, ĐHQG-HCM
KHOA TOÁN - TIN HỌC



HCMUS
Viet Nam National University
Ho Chi Minh City
University of Science

BÁO CÁO ĐỒ ÁN MÔN HỌC
NHẬN DẠNG MẪU (PATTERN RECOGNITION)

ĐỀ TÀI:

CÀI ĐẶT VÀ SO SÁNH CÁC MÔ HÌNH HỌC MÁY TRÊN TẬP DỮ LIỆU CHỮ SỐ VIẾT TAY MNIST

Giảng viên hướng dẫn: TS. Trương Phước Hưng
CN. Lê Phú Trường

Sinh viên thực hiện:
1. Trần Nguyên Hưng – MSSV: 24280048
2. Vương Thành Đạt – MSSV: 24280058
3. Phạm Phương Thảo – MSSV: 24280106

TP. HỒ CHÍ MINH, THÁNG 11 NĂM 2026

Mục lục

1 Giới thiệu bài toán và Hệ thống **2**
- 1.1 Đặt vấn đề và Mục tiêu . . . 2
- 1.2 Kiến trúc luồng xử lý (Pipeline) . . . 2

2 Khám phá và Tiền xử lý dữ liệu **2**
- 2.1 Phân tích khám phá dữ liệu (EDA) . . . 2
- 2.2 Chuẩn hoá Dữ liệu . . . 3

3 Mô hình 1: Multinomial Naïve Bayes **4**
- 3.1 Phương pháp và Quy trình xử lý . . . 4
- 3.2 Thí nghiệm, Đối chiếu Scikit-Learn và Đánh giá Kết quả . . . 5

4 Mô hình 2: One-vs-Rest Logistic Regression **6**
- 4.1 Phương pháp và Quy trình xử lý . . . 6
- 4.2 Thí nghiệm, Đối chiếu Scikit-Learn và Đánh giá Kết quả . . . 7

5 Mô hình 3: Multi-Layer Perceptron (MLP) và Convolutional Neural Network (CNN) **9**
- 5.1 Mạng nơ-ron truyền thẳng nhiều tầng (MLP) . . . 9
- 5.2 Thí nghiệm với các tham số MLP . . . 10
- 5.3 Mạng nơ-ron tích chập (CNN) và So sánh đánh giá . . . 11

6 Mô hình 4: Học quần thể (Ensemble) với AdaBoost **13**
- 6.1 Cơ sở lý thuyết thuật toán AdaBoost . . . 13
- 6.2 Khảo sát siêu tham số và đánh giá thực nghiệm AdaBoost . . . 14
  - 6.2.1 Khảo sát Accuracy theo thuật toán phân loại . . . 14
  - 6.2.2 Khảo sát siêu tham số . . . 14
  - 6.2.3 Ma trận nhầm lẫn trên tập kiểm thử . . . 14
  - 6.2.4 Nhận xét . . . 15

7 So sánh tổng hợp và Đánh giá các tiêu chí **15**
- 7.1 Bảng tổng hợp chỉ số metrics và thời gian chạy . . . 15
- 7.2 Diễn giải kết quả và đánh giá cơ bản các mô hình . . . 16
- 7.3 Phân tích và đưa ra kết luận mô hình tối ưu . . . 18

8 Kết luận và Hướng phát triển **19**
- 8.1 Kết luận . . . 19
- 8.2 Hạn chế và Hướng phát triển . . . 19

Phân công công việc nhóm và Khai báo sử dụng AI **20**

Tài liệu tham khảo **20**

1 Giới thiệu bài toán và Hệ thống

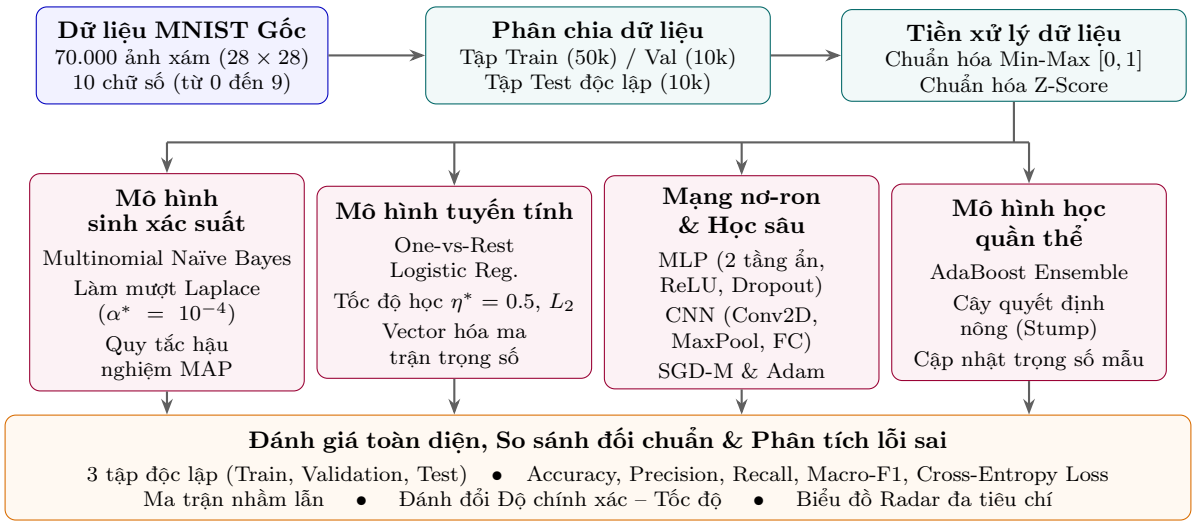
1.1 Đặt vấn đề và Mục tiêu

Bài toán nhận dạng chữ số viết tay trong đồ án này được thực nghiệm trên tập dữ liệu chuẩn MNIST. Bộ dữ liệu gồm 70.000 ảnh xám kích thước 28×28 pixel (10 nhãn từ 0 đến 9, cường độ $[0, 255]$), chia sẵn thành 60.000 mẫu huấn luyện và 10.000 mẫu kiểm thử. Đồ án tập trung giải quyết ba mục tiêu chính:

1. **Tiền xử lý dữ liệu:** Phân chia tập huấn luyện thành 50.000 mẫu train và 10.000 mẫu validation theo phương pháp phân tầng (stratified split), đồng thời chuẩn hóa đặc trưng phù hợp từng mô hình.
2. **Cài đặt mô hình:** Thực nghiệm 4 mô hình máy học:
   - Mô hình sinh xác suất: Multinomial Naïve Bayes.
   - Mô hình tuyến tính: One-vs-Rest Logistic Regression.
   - Mạng nơ-ron: Multi-Layer Perceptron (MLP) và Convolutional Neural Network (CNN).
   - Học quần thể (Ensemble): AdaBoost kết hợp bộ phân loại yếu.
3. **Đánh giá và so sánh:** Đo đạc hiệu năng qua Accuracy, F1-score, ma trận nhầm lẫn cùng thời gian huấn luyện và suy luận để đối sánh ưu nhược điểm giữa các phương pháp.

1.2 Kiến trúc luồng xử lý (Pipeline)

Quy trình thực nghiệm của nhóm được minh họa trực quan qua hình sau:



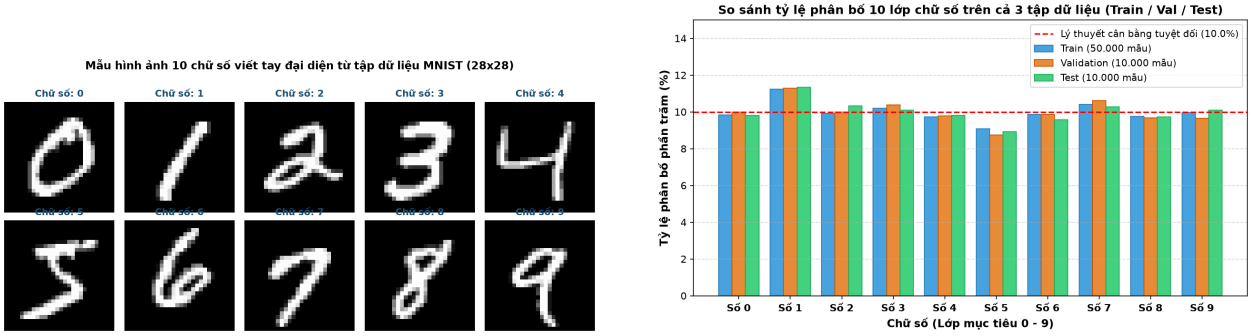
Hình 1: Quy trình tổng thể phân lớp chữ số viết tay MNIST qua 4 mô hình.

2 Khám phá và Tiền xử lý dữ liệu

2.1 Phân tích khám phá dữ liệu (EDA)

Trong bước phân tích khám phá dữ liệu (EDA), nhóm tiến hành thống kê phân phối của 60.000 mẫu huấn luyện và 10.000 mẫu kiểm thử gốc.

Để phục vụ việc đánh giá và tinh chỉnh mô hình mà không làm rò rỉ dữ liệu kiểm thử, tập huấn luyện được chia tách thành hai tập con: Train (50.000 mẫu) và Validation (10.000 mẫu). Quá trình phân chia được kiểm soát nhằm bảo toàn tỷ lệ xuất hiện đồng đều của 10 lớp chữ số trên cả hai tập, đồng thời đảm bảo khả năng tái lập kết quả thực nghiệm.



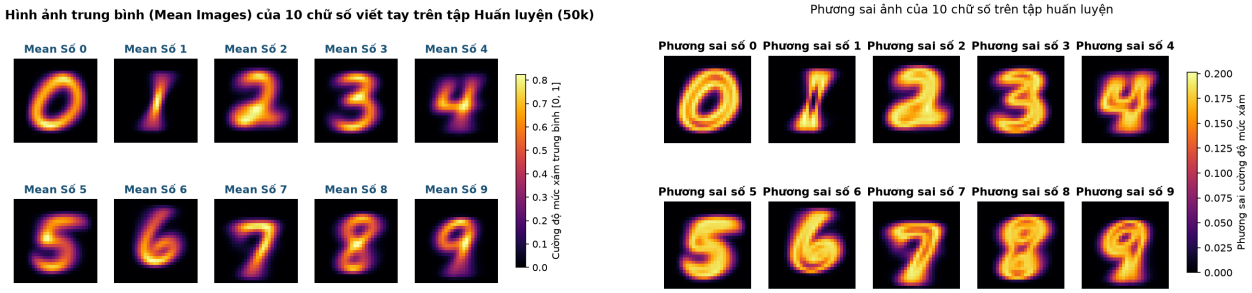
(a) Mẫu ngẫu nhiên 10 chữ số từ 0 đến 9.

(b) Phân bố tỷ lệ 10 lớp trên Train/Val/Test.

Hình 2: Trực quan hoá dữ liệu khám phá ban đầu trên tập dữ liệu MNIST.

Từ biểu đồ ở Hình 2, phân phối giữa các chữ số khá đồng đều, dao động từ 8.92% (số 5) đến 11.35% (số 1), cho thấy dữ liệu không bị mất cân bằng lớp. Bên cạnh đó, quá trình kiểm tra và thực nghiệm trên tập dữ liệu ghi nhận thêm hai đặc điểm:

- **Chất lượng dữ liệu:** Toàn bộ 60.000 mẫu không có giá trị khuyết thiếu (`NaN`) hay trường hợp trùng lặp.
- **Đặc trưng điểm ảnh:** Có 67/784 pixel nằm sát 4 cạnh viền luôn bằng 0 trên mọi mẫu (chiếm 8.55% diện tích), hầu như không mang giá trị phân loại.



(a) Ảnh trung bình (Mean Images) của 10 chữ số.

(b) Phương sai ảnh (Variance Images) của 10 chữ số.

Hình 3: Bản đồ nhiệt Ảnh trung bình và Phương sai ảnh của 10 chữ số trên tập Huấn luyện.

- **Ảnh trung bình (Mean):** Cho thấy hình dạng nét chữ đặc trưng của từng chữ số.
- **Phương sai ảnh (Variance):** Cho thấy mức độ biến thiên về độ nghiêng và nét bút của người viết.

Từ đó rút ra hai điểm trọng tâm: một số chữ số có nét tương đồng dễ gây nhầm lẫn (như 4 với 9, hoặc 3, 5 với 8), và 67 pixel viền ngoài có phương sai bằng 0 khiến mô hình Gaussian Naïve Bayes không phù hợp do lỗi chia cho 0.

2.2 Chuẩn hoá Dữ liệu

1. **Chuẩn hoá Min-Max (thang đo $[0, 1]$):**

$$x_{\text{norm}} = \frac{x}{255.0} \tag{1}$$

Đưa giá trị cường độ điểm ảnh từ khoảng $[0, 255]$ về đoạn $[0, 1]$. Phương pháp này bảo toàn tính không âm ($x \geq 0$), đáp ứng điều kiện đầu vào cho Multinomial Naïve Bayes và giúp giải thuật tối ưu của Logistic Regression hội tụ tốt hơn.

2. **Chuẩn hoá Z-score (Standardization):**

$$x_{\text{std}} = \frac{x - \mu}{\sigma + \epsilon} \tag{2}$$

Biến đổi dữ liệu về phân phối có kỳ vọng xấp xỉ 0 và phương sai bằng 1 với μ và σ là giá trị trung bình và độ lệch chuẩn tính trên tập huấn luyện, ϵ là số thực dương rất nhỏ nhằm tránh lỗi chia cho 0. Cách chuẩn hoá này giúp ổn định biên độ lan truyền và tối ưu hoá gradient cho các mô hình mạng nơ-ron (MLP).

3 Mô hình 1: Multinomial Naïve Bayes

3.1 Phương pháp và Quy trình xử lý

Multinomial Naïve Bayes mô hình hóa mỗi bức ảnh như một tập hợp các hạt mực phân bố trên $d = 784$ pixel. Với giả định các pixel độc lập có điều kiện theo từng lớp chữ số, nhãn dự đoán được xác định theo quy tắc hậu nghiệm cực đại (MAP) trong miền Log:

$$\hat{y} = \arg \max_{c \in \{0,\dots,9\}} \left[\ln P(y = c) + \sum_{j=1}^{d} x_j \ln \theta_{cj} \right] \tag{3}$$

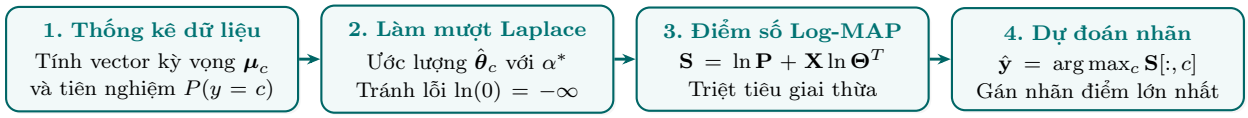
Trong đó $P(y = c)$ là xác suất tiên nghiệm của lớp c, θ_{cj} là xác suất hạt mực rơi vào pixel thứ j của chữ số c. Số hạng giai thừa trong phân phối đa thức độc lập với c nên triệt tiêu khi lấy arg max, giúp tránh tràn số và tối ưu tốc độ tính toán.

Do các pixel ở viền ảnh luôn bằng 0, ước lượng xác suất thuần túy sẽ dẫn đến $\theta_{cj} = 0 \implies \ln(0) = -\infty$. Để khắc phục, nhóm áp dụng kỹ thuật làm mượt Laplace với hệ số $\alpha > 0$:

$$\hat{\theta}_{cj} = \frac{\mu_{cj} + \alpha}{\sum_{k=1}^{d} \mu_{ck} + d \cdot \alpha} \tag{4}$$

với μ_{cj} là cường độ trung bình của pixel j trong lớp c, và mẫu số cộng thêm $d \cdot \alpha$ để bảo toàn điều kiện chuẩn hóa xác suất $\sum_{j=1}^{d} \hat{\theta}_{cj} = 1$.

Quy trình xử lý (Pipeline):



Thuật toán huấn luyện và suy luận Multinomial Naïve Bayes được mô tả trong Thuật toán 1.

Thuật toán 1 Huấn luyện và Dự đoán Multinomial Naïve Bayes

Đầu vào: Tập huấn luyện $\mathbf{X} \in [0,1]^{N \times d}$, nhãn $\mathbf{y} \in \{0,\ldots,9\}^N$, hệ số α, tập kiểm thử $\mathbf{X}_{\text{test}}$.
Đầu ra: Vector nhãn dự đoán $\hat{\mathbf{y}}_{\text{test}}$.

// Giai đoạn huấn luyện (Hàm fit):

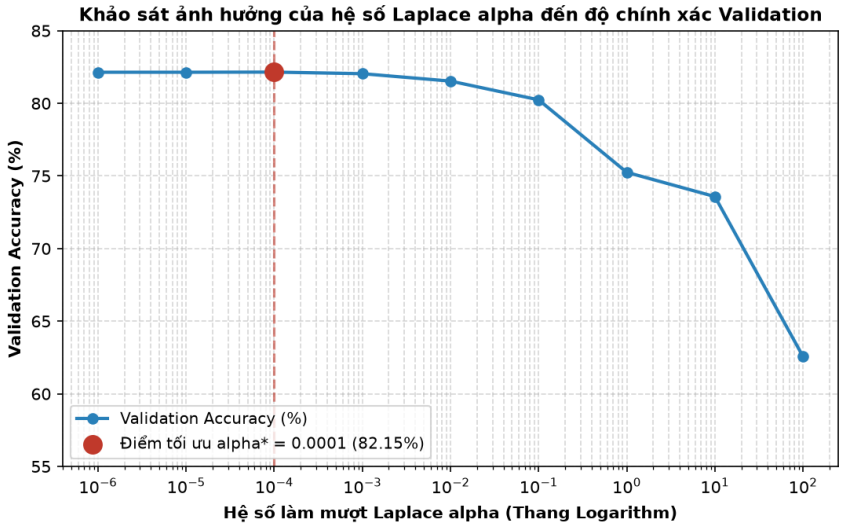
for mỗi lớp $c \in \{0,1,\ldots,9\}$ **do**
  Lọc các mẫu thuộc lớp c: $\mathbf{X}_c \leftarrow \mathbf{X}[\mathbf{y} == c]$
  Xác suất tiên nghiệm: $\text{prior}[c] \leftarrow \frac{|\mathbf{X}_c|}{N}$
  Cường độ trung bình từng pixel: $\boldsymbol{\mu}_c \leftarrow \frac{1}{|\mathbf{X}_c|} \sum_{\mathbf{x} \in \mathbf{X}_c} \mathbf{x}$
end for

// Giai đoạn dự đoán (Hàm predict_log_proba & predict):

for mỗi lớp $c \in \{0,1,\ldots,9\}$ **do**
  Xác suất rơi mực Laplace: $\boldsymbol{\theta}_c \leftarrow \frac{\boldsymbol{\mu}_c + \alpha}{\sum_{j=1}^{d}(\mu_{cj} + \alpha)}$
  Log-likelihood của N_{test} mẫu: $\mathbf{L}_c \leftarrow \sum_{j=1}^{d} x_{ij} \ln \theta_{cj}$
  Điểm số hậu nghiệm: $\mathbf{S}[:, c] \leftarrow \mathbf{L}_c + \ln \text{prior}[c]$
end for
Gán nhãn có điểm lớn nhất: $\hat{\mathbf{y}}_{\text{test}} \leftarrow \arg\max_{c \in \{0,\ldots,9\}} \mathbf{S}[:, c]$
return $\hat{\mathbf{y}}_{\text{test}}$

3.2 Thí nghiệm, Đối chiếu Scikit-Learn và Đánh giá Kết quả

Khảo sát siêu tham số làm mượt Laplace α: Nhóm tiến hành khảo sát lưới siêu tham số trên thang đo Logarit $\alpha \in [10^{-6}, 10^{-5}, 10^{-4}, 10^{-3}, 10^{-2}, 0.1, 1.0, 10.0, 100.0]$ trên tập Validation (10.000 mẫu). Kết quả trên Hình 4 cho thấy điểm cực trị đạt được tại $\alpha^* = 10^{-4}$ với độ chính xác cao nhất đạt 82.15% trên tập Validation và 83.56% trên tập Test. Khi tăng $\alpha \geq 1.0$, độ chính xác suy giảm rõ rệt và tụt xuống 62.57% khi $\alpha = 100.0$ do hiện tượng làm mượt quá mức làm san phẳng phân phối đặc trưng (Underfitting).

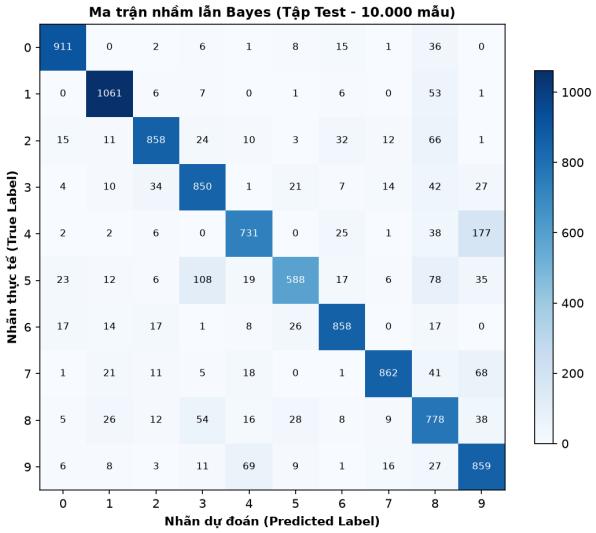


Hình 4: Khảo sát siêu tham số Laplace smoothing α trên tập Validation (đỉnh tối ưu tại $\alpha^* = 10^{-4}$).

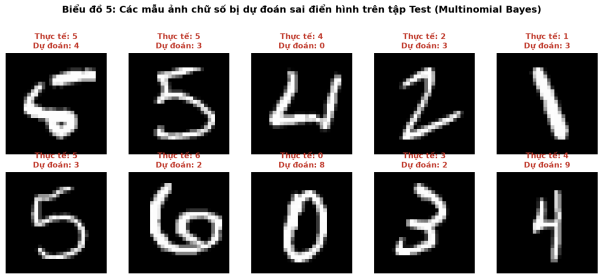
Đối chiếu Scikit-Learn: Kiểm chứng trực tiếp với mô hình `MultinomialNB` của thư viện Scikit-Learn ở cùng giá trị $\alpha^* = 10^{-4}$ cho thấy mô hình tự cài đặt đạt độ chính xác 83.56%, bám sát kết quả của thư viện (83.63%, chênh lệch chỉ 0.07%) và tỷ lệ đồng thuận dự đoán đạt trên 99.8%, khẳng định tính chính xác của thuật toán cài đặt.

Bảng 1: Kết quả thực nghiệm của Multinomial Naïve Bayes ($\alpha^* = 10^{-4}$).

Tập dữ liệu	Số mẫu	Accuracy (%)	Precision (%)	Recall (%)	Macro F1 (%)	Thời gian (s)
Train	50.000	82.47	83.19	82.18	82.35	**0.06**
Validation	10.000	82.15	82.77	81.80	81.98	**0.03**
Test	10.000	**83.56**	84.24	83.25	83.41	**0.03**



(a) MNB - Confushion Test (Accuracy = 83.56%).



(b) Các mẫu ảnh bị nhận diện sai điển hình.

Hình 5: Ma trận nhầm lẫn và các trường hợp lỗi phân loại của Multinomial Naïve Bayes.

Nhận xét kết quả thực nghiệm:

- **Hiệu năng tính toán:** Mô hình có tốc độ huấn luyện rất cao (0.06 giây cho 50.000 mẫu) và độ trễ suy luận chỉ khoảng 0.03 ms/mẫu. Điều này đạt được do thuật toán chỉ thực hiện ước lượng tham số qua đếm tần suất thống kê thay vì tối ưu hóa lặp đi lặp lại.
- **Độ chính xác theo lớp:** Nhận diện tốt nhất ở chữ số 0 ($F_1 = 92.77\%$) và 1 ($F_1 = 92.26\%$) nhờ cấu trúc nét đơn giản, ít trùng lặp. Ngược lại, số 5 có Recall thấp nhất (65.92%, bỏ sót 304 mẫu) và số 8 có Precision thấp nhất (66.16%, có 398 mẫu bị phân loại nhầm thành số 8).
- **Phân tích lỗi và nguyên nhân:** Sự nhầm lẫn chủ yếu ở cặp số (4, 9) và cặp ($5 \to 3$). Nguyên nhân do giả định độc lập có điều kiện ($x_j \perp x_k \mid y = c$) bỏ qua tương quan không gian giữa các pixel, khiến mô hình chỉ dựa vào phân bố cường độ điểm ảnh riêng lẻ mà không nắm bắt được cấu trúc nét vẽ liền mạch.

4 Mô hình 2: One-vs-Rest Logistic Regression

4.1 Phương pháp và Quy trình xử lý

Logistic Regression là mô hình phân biệt tuyến tính, tối ưu hóa trực tiếp xác suất phân lớp của mẫu dữ liệu. Để giải quyết bài toán nhận dạng 10 chữ số, nhóm áp dụng chiến lược One-vs-Rest, huấn luyện đồng thời 10 bộ phân loại nhị phân tương ứng với 10 lớp ($c \in \{0, \ldots, 9\}$).

Với mỗi mẫu ảnh được chèn bias $\tilde{\mathbf{x}} = [\mathbf{x}, 1] \in \mathbb{R}^{785}$ và vector trọng số $\mathbf{w}_c \in \mathbb{R}^{785}$, điểm số tuyến tính $z = \tilde{\mathbf{x}}\mathbf{w}_c$ được chuyển thành xác suất thông qua hàm Sigmoid phân nhánh để chống tràn số:

$$\sigma(z) = \begin{cases} \dfrac{1}{1+e^{-z}}, & z \geq 0 \\ \dfrac{e^{z}}{1+e^{z}}, & z < 0 \end{cases} \tag{5}$$

Hàm mất mát Binary Cross-Entropy kết hợp điều chuẩn L_2 cho từng chữ số c có dạng:

$$J(\mathbf{w}_c) = \frac{1}{N}\sum_{i=1}^{N}\Big[\ln(1+e^{z_{ic}}) - t_{ic}z_{ic}\Big] + \frac{\lambda}{2}\|\mathbf{m}\odot\mathbf{w}_c\|^2 \tag{6}$$

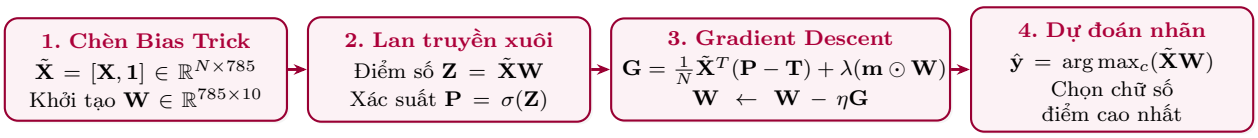
với $z_{ic} = \tilde{\mathbf{x}}_i\mathbf{w}_c$ và $t_{ic} \in \{0,1\}$ là nhãn One-hot của mẫu i đối với chữ số c. Biểu thức $\ln(1+e^z)$ được tính toán bằng `np.logaddexp(0, z)` giúp triệt tiêu hoàn toàn nguy cơ tràn số. Vector mặt nạ $\mathbf{m} = [1,\ldots,1,0]^T \in \mathbb{R}^{785}$ đảm bảo không phạt điều chuẩn lên hệ số tự do w_0, giúp các siêu phẳng tự do dịch chuyển tịnh tiến mà không làm méo ranh giới phân lớp.

Kỹ thuật Vector hóa ma trận: Thay vì lặp tuần tự 10 bộ phân loại, nhóm gộp toàn bộ tham số thành ma trận trọng số $\mathbf{W} \in \mathbb{R}^{785\times 10}$ và ma trận nhãn $\mathbf{T} \in \{0,1\}^{N\times 10}$. Toàn bộ quá trình tính đạo hàm Gradient được thực hiện song song bằng một phép nhân ma trận:

$$\nabla_{\mathbf{W}}J = \frac{1}{N}\tilde{\mathbf{X}}^T(\mathbf{P}-\mathbf{T}) + \lambda(\mathbf{m}\odot\mathbf{W}), \quad \mathbf{W} \leftarrow \mathbf{W} - \eta\nabla_{\mathbf{W}}J \tag{7}$$

Trong đó $\mathbf{P} = \sigma(\tilde{\mathbf{X}}\mathbf{W}) \in [0,1]^{N\times 10}$ là ma trận xác suất dự đoán, và η là tốc độ học.

Quy trình xử lý:



Thuật toán One-vs-Rest Logistic Regression được mô tả chi tiết trong Thuật toán 2.

Thuật toán 2 Huấn luyện và Dự đoán One-vs-Rest Logistic Regression

Đầu vào: Tập huấn luyện $\mathbf{X} \in \mathbb{R}^{N\times d}$, nhãn $\mathbf{y} \in \{0,\ldots,9\}^N$, tốc độ học η, hệ số λ, số epoch `max_epochs`, tập kiểm thử $\mathbf{X}_{\text{test}}$.

Đầu ra: Vector nhãn dự đoán $\hat{\mathbf{y}}_{\text{test}}$.

// Giai đoạn huấn luyện (Hàm fit):

Thêm cột bias: $\tilde{\mathbf{X}} \leftarrow [\mathbf{X}, \mathbf{1}] \in \mathbb{R}^{N\times(d+1)}$
Tạo ma trận nhãn One-hot: $\mathbf{T} \in \{0,1\}^{N\times 10}$ với $\mathbf{T}[:,c] \leftarrow (\mathbf{y} == c)$
Khởi tạo ma trận trọng số $\mathbf{W} \leftarrow \mathbf{0} \in \mathbb{R}^{(d+1)\times 10}$, mặt nạ không phạt bias $\mathbf{m} = [1,\ldots,1,0]^T$
for epoch $= 1 \rightarrow$ max_epochs **do**
    Ma trận điểm số: $\mathbf{Z} \leftarrow \tilde{\mathbf{X}}\mathbf{W}$
    Ma trận xác suất: $\mathbf{P} \leftarrow \sigma(\mathbf{Z})$
    Ma trận Gradient: $\mathbf{G} \leftarrow \frac{1}{N}\tilde{\mathbf{X}}^T(\mathbf{P}-\mathbf{T}) + \lambda(\mathbf{m}\odot\mathbf{W})$
    Cập nhật song song 10 bộ trọng số: $\mathbf{W} \leftarrow \mathbf{W} - \eta\mathbf{G}$
end for

// Giai đoạn dự đoán (Hàm predict):

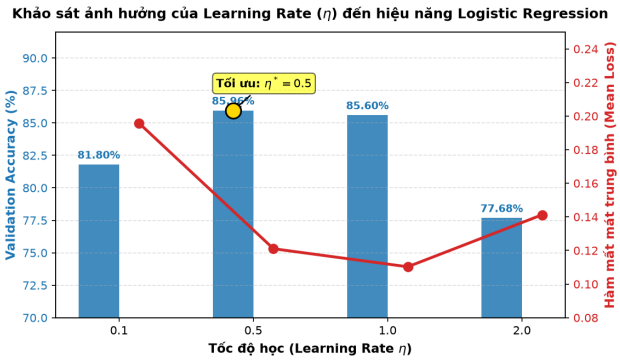
Thêm bias tập kiểm thử: $\tilde{\mathbf{X}}_{\text{test}} \leftarrow [\mathbf{X}_{\text{test}}, \mathbf{1}]$
Tính ma trận điểm số: $\mathbf{Z}_{\text{test}} \leftarrow \tilde{\mathbf{X}}_{\text{test}}\mathbf{W}$
Gán nhãn có điểm cao nhất: $\hat{\mathbf{y}}_{\text{test}} \leftarrow \arg\max_{c\in\{0,\ldots,9\}} \mathbf{Z}_{\text{test}}[:,c]$
return $\hat{\mathbf{y}}_{\text{test}}$

4.2 Thí nghiệm, Đối chiếu Scikit-Learn và Đánh giá Kết quả

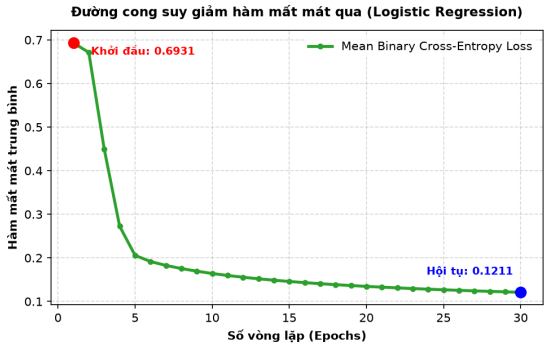
Khảo sát siêu tham số và Động học hội tụ:

1. **Quét tốc độ học** $\eta \in [0.1, 0.5, 1.0, 2.0]$**:** Khảo sát trên tập Validation (10.000 mẫu, 30 epoch) trên Hình 6a xác định điểm cực trị đạt tại $\eta^* = 0.5$ với độ chính xác cao nhất đạt 85.96% và hàm mất mát 0.1211. Mức $\eta = 0.1$ hội tụ chậm (81.80%), trong khi $\eta = 2.0$ bước nhảy quá lớn gây dao động mạnh làm suy giảm hiệu năng (77.68%). Do đó, nhóm lựa chọn $\eta^* = 0.5$ làm tham số tối ưu để huấn luyện mô hình chính thức.

2. **Đường cong suy giảm hàm mất mát:** Qua 30 epoch huấn luyện chính thức (Hình 6b), hàm mất mát giảm từ 0.6931 và phẳng dần về đáy (0.1211), khẳng định thuật toán Gradient Descent hội tụ ổn định, không xảy ra hiện tượng quá khớp.

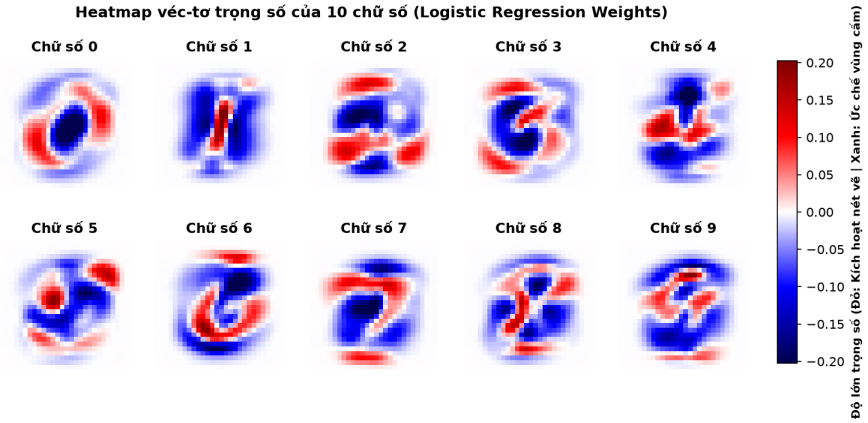


(a) Khảo sát tốc độ học η trên tập Validation.



(b) Đường cong suy giảm hàm mất mát.

Hình 6: Thực nghiệm tinh chỉnh siêu tham số và động học hội tụ của Logistic Regression.



Hình 7: Bản đồ nhiệt ma trận trọng số $\mathbf{W}$ (28×28) của 10 chữ số.

Phân tích bản đồ nhiệt trọng số (Hình 7): Mỗi lớp chữ số tương ứng với một ma trận trọng số 28×28, phản ánh cơ chế ra quyết định của mô hình:

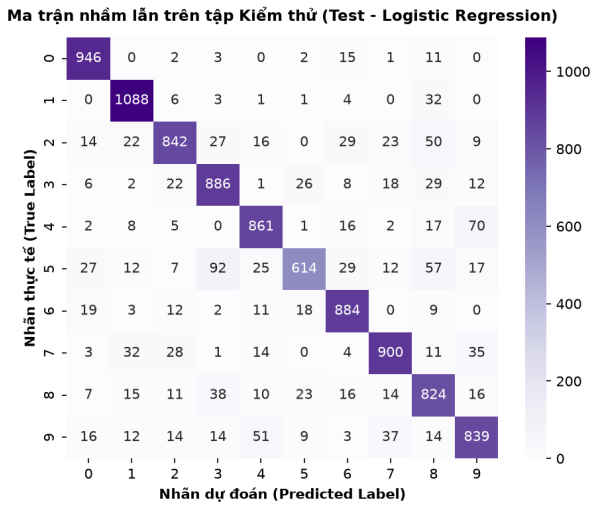
- **Vùng trọng số dương ($w_j > 0$):** Đóng vai trò kích hoạt. Điểm ảnh có nét mực rơi vào vùng này sẽ làm tăng xác suất dự đoán (ví dụ: viền ngoài của số 0, nét thẳng của số 1).
- **Vùng trọng số âm ($w_j < 0$):** Đóng vai trò ức chế. Nét mực xuất hiện tại đây sẽ làm giảm mạnh điểm số của lớp (ví dụ: phần tâm rỗng của số 0 mang trọng số âm lớn, giúp phân biệt rõ với các số có nét cắt qua tâm như 1, 4, 8).

Ngoài ra, điều chuẩn L_2 ($\lambda = 10^{-4}$) giúp trường trọng số phân bổ liền mạch, hạn chế hiện tượng nhiễu cục bộ giữa các pixel lân cận.

Đối chiếu Scikit-Learn: Kiểm chứng với `OneVsRestClassifier` từ thư viện Scikit-Learn cho kết quả tương đương với Test Accuracy đạt 90.59% (so với 86.84% của mô hình tự cài đặt) và tỷ lệ đồng thuận dự đoán đạt **89.62%**, khẳng định tính tin cậy của thuật toán cài đặt.

Bảng 2: Kết quả thực nghiệm của One-vs-Rest Logistic Regression ($\eta^* = 0.5$, 30 epochs).

Tập dữ liệu	Số mẫu	Accuracy (%)	Precision (%)	Recall (%)	Macro F1 (%)	Thời gian (s)
Train	50.000	86.00	86.04	85.72	85.71	10.70
Validation	10.000	85.96	85.96	85.64	85.69	**0.03**
Test	10.000	**86.84**	86.87	86.58	86.55	**0.03**



(a) LR (OvR) Confushion Test (Acc = 86.84%).



(b) Các mẫu ảnh bị nhận diện sai điển hình.

Hình 8: Ma trận nhầm lẫn và các trường hợp lỗi phân loại của Logistic Regression.

Nhận xét kết quả thực nghiệm:

- **Hiệu năng tổng thể:** Độ chính xác trên tập kiểm thử đạt **86.84%** (tăng 3.28% so với Naïve Bayes) nhờ cơ chế tối ưu phân biệt và khả năng ức chế bằng trọng số âm. Thời gian huấn luyện là 10.70s và độ trễ suy luận đạt 0.003 ms/mẫu.
- **Kết quả theo từng lớp:** Mô hình phân loại tốt nhất ở số 0 ($F_1 = 93.66\%$) và số 1 ($F_1 = 93.43\%$). Khó phân biệt nhất là số 5 do Recall thấp (68.83%, bỏ sót 278 mẫu) và số 8 do Precision thấp (78.18%).
- **Phân tích lỗi và hạn chế:** Các nhầm lẫn phổ biến nhất gồm ($5 \to 3$) với 92 mẫu, ($4 \leftrightarrow 9$) với 121 mẫu, cùng các cặp ($5 \to 8$) và ($2 \to 8$). Hạn chế cốt lõi nằm ở ranh giới quyết định tuyến tính, không thể nắm bắt được độ nghiêng và cấu trúc phi tuyến phức tạp (Hình 8b), dẫn đến nhu cầu áp dụng các mô hình phi tuyến như **MLP** và **CNN**.

5 Mô hình 3: Multi-Layer Perceptron (MLP) và Convolutional Neural Network (CNN)

5.1 Mạng nơ-ron truyền thẳng nhiều tầng (MLP)

Mô hình Multi-Layer Perceptron (MLP) giải quyết bài toán phân loại chữ số bằng cách kết hợp các phép biến đổi tuyến tính với hàm kích hoạt phi tuyến ReLU. Nhóm sử dụng kiến trúc baseline $784 \to 256 \to 128 \to 10$, trong đó ma trận trọng số được khởi tạo theo phân phối He.

Ở chiều lan truyền thuận, dữ liệu đi qua các tầng ẩn kết hợp kỹ thuật Inverted Dropout để chống quá khớp. Tại tầng ra, các điểm số logits được chuẩn hóa thành xác suất dự đoán $\hat{\mathbf{Y}}$ qua hàm Softmax.

Quá trình lan truyền ngược tối ưu hàm mất mát Cross-Entropy bắt đầu từ sai số tầng ra:

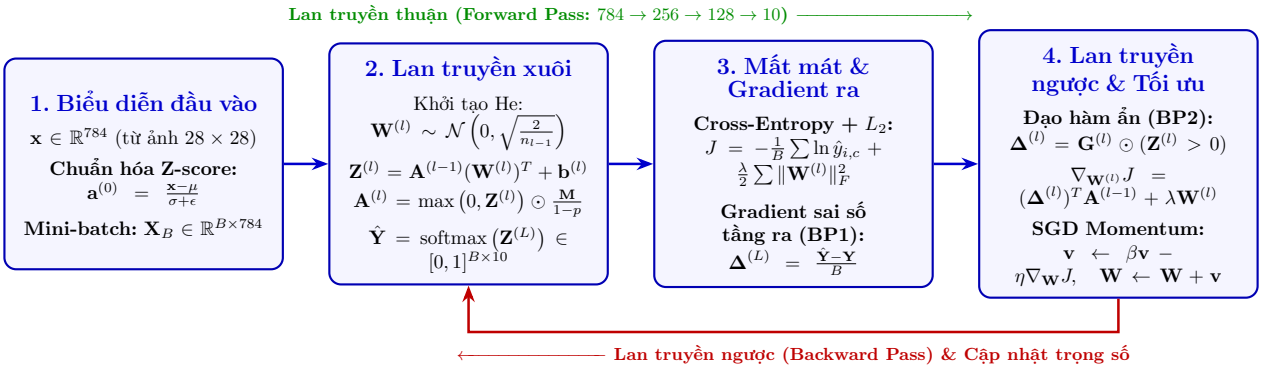
$$\boldsymbol{\Delta}^{(L)} = \frac{\hat{\mathbf{Y}} - \mathbf{Y}}{N} \tag{8}$$

Tín hiệu sai số được truyền ngược lần lượt qua các tầng ẩn theo đạo hàm ReLU. Nhờ kỹ thuật vector hóa ma trận, gradient của trọng số và bias tại mỗi tầng được tính toán song song kết hợp điều chuẩn L_2 ($\lambda = 10^{-4}$):

$$\frac{\partial J_{\text{reg}}}{\partial \mathbf{W}^{(l)}} = (\boldsymbol{\Delta}^{(l)})^T \mathbf{A}^{(l-1)} + \lambda \mathbf{W}^{(l)}, \quad \frac{\partial J}{\partial \mathbf{b}^{(l)}} = \sum_{i=1}^{N} \boldsymbol{\Delta}_i^{(l)} \tag{9}$$

Mạng được huấn luyện theo từng mini-batch bằng giải thuật SGD có động lượng (Momentum). Đồng thời, cơ chế Early Stopping được áp dụng để theo dõi `val_loss`: nếu sau 4 epoch liên tiếp mất mát kiểm định không cải thiện, quá trình huấn luyện sẽ dừng sớm và khôi phục trạng thái trọng số tối ưu nhất.

Quy trình xử lý (Pipeline):



Thuật toán MLP được mô tả chi tiết trong Thuật toán 3.

Thuật toán 3 Huấn luyện Mạng nơ-ron nhiều tầng (MLP Backpropagation with Momentum)

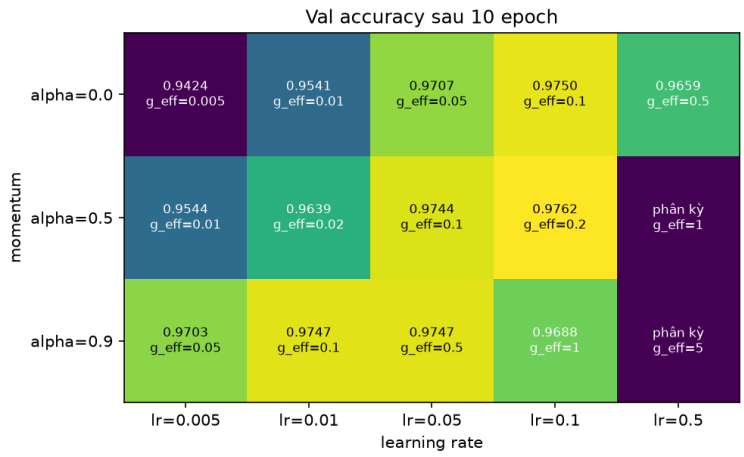
Đầu vào: Dữ liệu $\mathbf{X}$, nhãn $\mathbf{y}$, số tầng L, tốc độ học η, hệ số quán tính β, batch size B.
Đầu ra: Tập trọng số và độ lệch tối ưu $\{\mathbf{W}^{(l)}, \mathbf{b}^{(l)}\}_{l=1}^{L}$.

1: Khởi tạo trọng số He $\mathbf{W}^{(l)} \sim \mathcal{N}(0, \sqrt{2/n_{l-1}})$, bias $\mathbf{b}^{(l)} \leftarrow \mathbf{0}$, vận tốc $\mathbf{v}^{(l)} \leftarrow \mathbf{0}$.
2: **for** mỗi mini-batch $(\mathbf{X}_B, \mathbf{Y}_B)$ **do**
3: **Forward:** $\mathbf{z}^{(l)} = \mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$, $\mathbf{a}^{(l)} = \mathrm{ReLU}(\mathbf{z}^{(l)})$; tầng cuối Softmax $\mathbf{P} = \mathrm{softmax}(\mathbf{z}^{(L)})$.
4: **Backward:** Lỗi tầng ra $\boldsymbol{\delta}^{(L)} = \mathbf{P} - \mathbf{T}$; lỗi tầng ẩn $\boldsymbol{\delta}^{(l)} = [(\mathbf{W}^{(l+1)})^T \boldsymbol{\delta}^{(l+1)}] \odot \mathbf{1}(\mathbf{z}^{(l)} > 0)$.
5: **Update:** $\mathbf{v}^{(l)} \leftarrow \beta\mathbf{v}^{(l)} + \eta\left(\frac{1}{B}\boldsymbol{\delta}^{(l)}(\mathbf{a}^{(l-1)})^T + \lambda\mathbf{W}^{(l)}\right)$, $\mathbf{W}^{(l)} \leftarrow \mathbf{W}^{(l)} - \mathbf{v}^{(l)}$.
6: **end for**
7: **return** $\{\mathbf{W}^{(l)}, \mathbf{b}^{(l)}\}_{l=1}^{L}$

5.2 Thí nghiệm với các tham số MLP

Từ mô hình baseline ban đầu ($784 \to 256 \to 128 \to 10$, ReLU, khởi tạo He, $\gamma = 0.05$, $\alpha = 0.9$, batch size 128, đạt Val Accuracy 97.77%), nhóm tiến hành thực nghiệm đơn biến trên tập Validation (10 epoch/cấu hình) để xác định bộ siêu tham số tối ưu.

a. Tốc độ học (γ) và Quán tính (α): Khảo sát lưới $\gamma \in \{0.005, 0.01, 0.05, 0.1, 0.5\}$ và $\alpha \in \{0.0, 0.5, 0.9\}$ với tốc độ học hiệu dụng $\gamma_{\text{eff}} = \frac{\gamma}{1-\alpha}$.



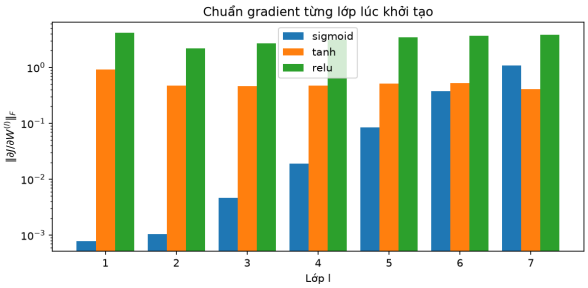
Hình 9: Khảo sát Learning rate và Momentum

b. Kích thước Mini-batch và Kiến trúc mạng: Cố định cặp tham số tốt nhất từ bước 1 ($\gamma = 0.1, \alpha = 0.5$), nhóm lần lượt khảo sát kích thước batch và 5 cấu hình độ rộng/sâu của mạng:

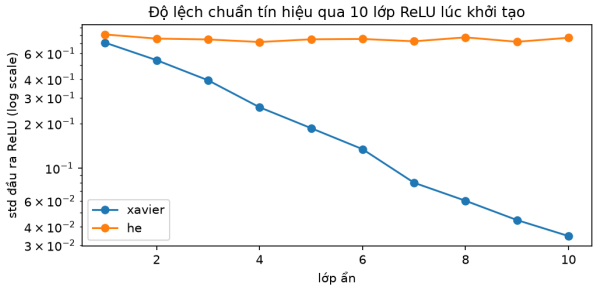
Batch	Cập nhật	Giây/ep	Val Acc
16	15.625	3.97s	96.11%
64	**3.910**	**1.57s**	**97.94%**
256	980	0.72s	97.37%
1024	245	0.49s	96.60%

Kiến trúc mạng	Tham số	Train	Val Acc
$784 \to 64 \to 10$	50.890	99.37%	97.12%
$784 \to 256 \to 10$	203.530	100%	97.63%
$784 \to 1024 \to 10$	814.090	100%	97.90%
$\mathbf{784 \to 256 \to 128 \to 10}$	**235.146**	**100%**	**97.94%**
$784 \to 256^4 \to 10$	400.906	99.38%	97.30%

c. Hàm kích hoạt và Khởi tạo trọng số: Đo chuẩn Frobenius $\|\partial J/\partial \mathbf{W}^{(l)}\|_F$ qua 6 lớp ẩn ($784 \to 128^6 \to 10$) và độ lệch chuẩn tín hiệu (std) qua 10 lớp ReLU ($784 \to 256^{10} \to 10$):



Hình 10: Chuẩn gradient theo hàm kích hoạt



Hình 11: Độ lệch chuẩn tín hiệu qua 10 lớp ReLU

Nhận xét và Lựa chọn cấu hình tối ưu:

- **Siêu tham số huấn luyện:** Cặp ($\gamma = 0.1, \alpha = 0.5$) giúp hội tụ ổn định, tránh hiện tượng phân kỳ (`NaN`) khi $\gamma = 0.5, \alpha \geq 0.5$. Kết hợp cùng **batch size 64**, mô hình đạt độ chính xác kiểm định cao nhất (**97.94%**) với tốc độ hợp lý (1.57s/epoch).
- **Kiến trúc và Khởi tạo:** Cấu trúc $\mathbf{784 \to 256 \to 128 \to 10}$ (235.146 tham số) cho hiệu năng tốt hơn mạng rộng 1024 nút và mạng sâu 4 lớp ẩn. Việc kết hợp **ReLU và khởi tạo He** giúp khắc phục triệt để hiện tượng tiêu biến gradient của Sigmoid (suy giảm 1.400 lần) và suy giảm phương sai của Xavier.

5.3 Mạng nơ-ron tích chập (CNN) và So sánh đánh giá

Khác với MLP làm phẳng ảnh khiến cấu trúc lân cận bị phá vỡ, mạng tích chập (CNN) giữ nguyên đầu vào dạng tensor 2D $\mathbf{X} \in [0,1]^{1\times 28\times 28}$ để khai thác trực tiếp tương quan không gian cục bộ (nét bút, góc cạnh).

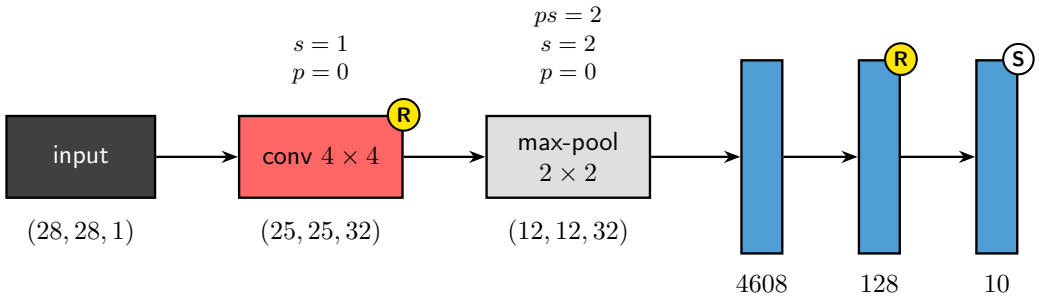
Tại tầng tích chập (Conv2D), 32 bộ lọc $\mathbf{K}_k$ kích thước 4×4 ($s = 1, p = 0$) trượt trên ảnh để trích xuất bản đồ đặc trưng và kích hoạt qua hàm ReLU:

$$z_{k,i,j} = \sum_{c,u,v} K_{k,c,u,v}\, x_{c,i+u,j+v} + b_k, \qquad a_{k,i,j} = \mathrm{ReLU}(z_{k,i,j}) \tag{10}$$

Tiếp theo, tầng gộp cực đại (Max-Pooling $2 \times 2, s = 2$) giữ lại đặc trưng nổi bật nhất trong từng vùng cục bộ, giúp giảm một nửa kích thước không gian và tăng tính bất biến dịch chuyển:

$$p_{k,i,j} = \max_{(u,v)\in\{0,1\}^2} a_{k,2i+u,2j+v} \tag{11}$$

Bản đồ đặc trưng ($12 \times 12 \times 32$) sau đó được trải phẳng thành vector 4608 chiều, đi qua tầng kết nối đầy đủ (Dense 128 + ReLU) và chuẩn hóa bằng hàm Softmax ở tầng ra để dự đoán chữ số $\hat{y} = \arg\max_k P(y = k \mid \mathbf{x})$. Toàn bộ mạng gồm 591.786 tham số, được huấn luyện bằng bộ tối ưu Adam ($\mathrm{lr} = 10^{-3}$, batch size 128) kết hợp Early Stopping (patience = 2).



Hình 12: Kiến trúc mạng tích chập `MNISTCNN` (**R**: kích hoạt ReLU, **S**: hàm Softmax)

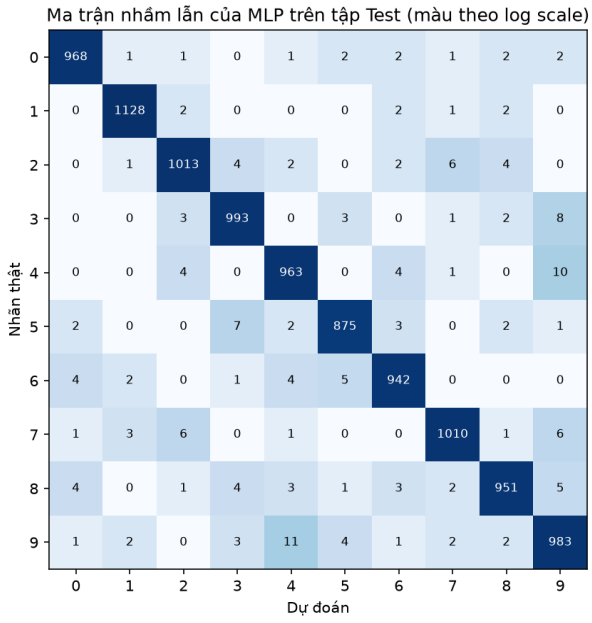
Trong khuôn khổ nghiên cứu này, việc triển khai mạng nơ-ron tích chập (CNN) được xem như một bước tiếp cận mở rộng nhằm làm rõ bản chất hoạt động của mô hình. Nhóm quyết định thiết lập CNN ở cấu hình cơ bản để tạo ra một hệ quy chiếu (baseline) khách quan phục vụ cho công tác đối chứng. Quá trình tinh chỉnh siêu tham số (hyperparameter tuning) chuyên sâu nhằm tối ưu hóa hiệu năng sẽ là định hướng phát triển ở các giai đoạn tiếp theo.

Bảng 3: Kết quả thực nghiệm của Mạng nơ-ron nhiều tầng (MLP).

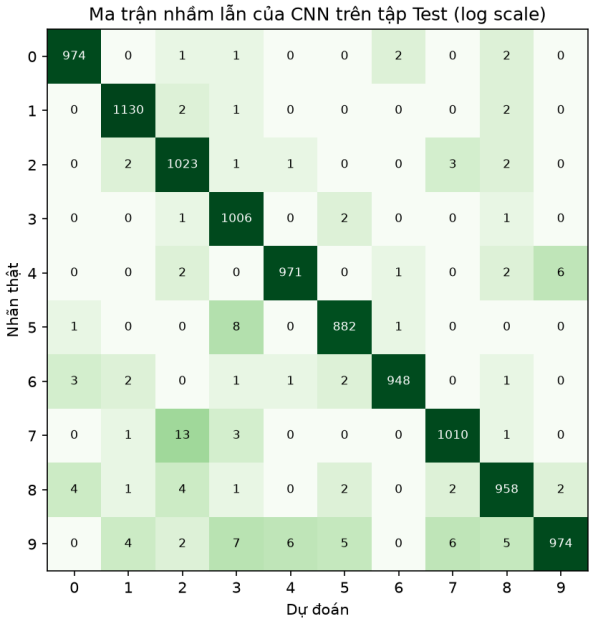
Tập dữ liệu	Số mẫu	Accuracy (%)	Precision (%)	Recall (%)	Macro F1 (%)	Thời gian (s)
Train	50.000	100.00	100.00	100.00	100.00	33.57
Validation	10.000	97.88	97.85	97.87	97.86	**0.02**
Test	10.000	**98.26**	98.25	98.24	98.25	**0.02**

Bảng 4: Kết quả thực nghiệm của Mạng nơ-ron tích chập (CNN).

Tập dữ liệu	Số mẫu	Accuracy (%)	Precision (%)	Recall (%)	Macro F1 (%)	Thời gian (s)
Train	50.000	99.65	99.65	99.65	99.65	51.61
Validation	10.000	98.58	98.57	98.57	98.57	0.44
Test	10.000	**98.76**	98.77	98.75	98.76	**0.43**

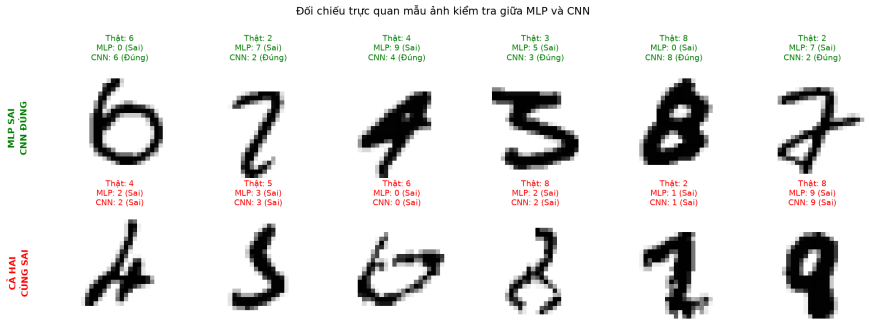


(a) Ma trận nhầm lẫn MLP (Test Acc = 97.56%).



(b) Ma trận nhầm lẫn CNN (Test Acc = 98.76%).

Hình 13: Đối chiếu ma trận nhầm lẫn trên tập Test của MLP và CNN.



Hình 14: Phân tích các mẫu chữ viết tay mà MLP đoán sai nhưng CNN nhận diện chính xác.

So sánh hiệu năng giữa MLP và CNN:

- **Độ chính xác:** Trên tập Test (10.000 mẫu), CNN đạt Accuracy **98.76%** (124 mẫu sai), vượt trội hơn MLP với **98.26%** (174 mẫu sai).
- **Độ phức tạp và Tốc độ:** CNN sở hữu **591.786 tham số** (gấp 2.5 lần MLP) và thời gian huấn luyện dài hơn (44 giây so với 30 giây của MLP). Đáng chú ý, gần 99.7% tham số của CNN tập trung ở lớp Dense đầu tiên sau Flatten, trong khi lớp tích chập chỉ chiếm 544 tham số.

Tóm lại: Sự chênh lệch này là do cách hai mô hình xử lý dữ liệu. MLP yêu cầu duỗi thẳng ảnh thành một vector 1D, vô tình làm mất đi mối liên hệ không gian giữa các điểm ảnh. Trong khi đó, CNN giữ nguyên cấu trúc lưới 2D của ảnh và dùng các bộ lọc (filter) để trích xuất trực tiếp các đặc trưng cục bộ (đường nét, góc cạnh).

6 Mô hình 4: Học quần thể (Ensemble) với AdaBoost

6.1 Cơ sở lý thuyết thuật toán AdaBoost

AdaBoost kết hợp tuần tự nhiều bộ phân loại yếu thành mô hình tổng hợp. Sau mỗi vòng, các mẫu bị dự đoán sai được tăng trọng số để những vòng sau tập trung sửa lỗi. Thuật toán 4 trình bày SAMME, biến thể AdaBoost dùng để phân loại trực tiếp 10 lớp MNIST.

Thuật toán 4 Huấn luyện Học quần thể AdaBoost đa lớp (SAMME)

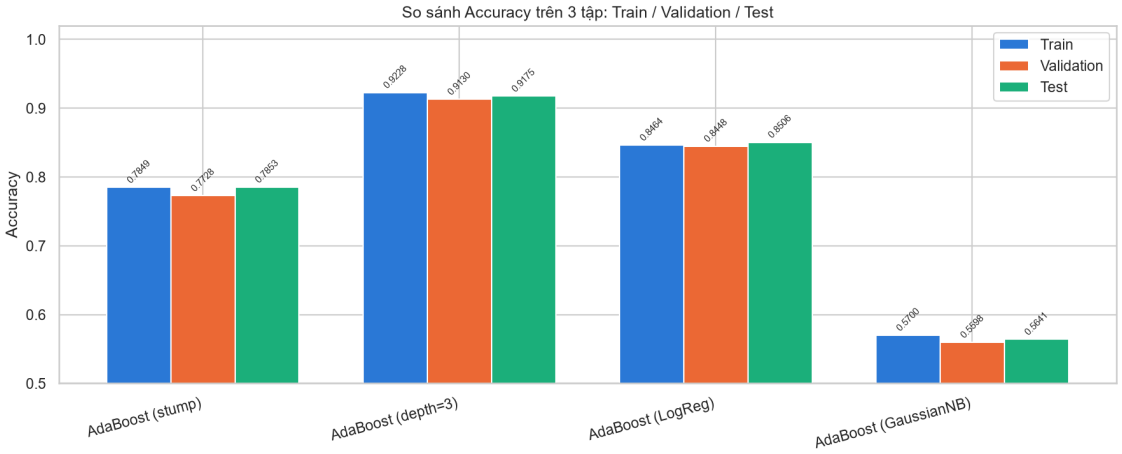
Đầu vào: Tập dữ liệu $\{(\mathbf{x}_i, y_i)\}_{i=1}^{N}$, số lớp K, số vòng T, tốc độ học η.
Đầu ra: Bộ phân loại tổng hợp $H(\mathbf{x}) = \arg\max_k \sum_{t=1}^{T'} \alpha_t \mathbf{1}[h_t(\mathbf{x}) = k]$.

Khởi tạo trọng số mẫu phân bố đều: $w_1(i) = 1/N$, $\forall i \in \{1, \ldots, N\}$.
for $t = 1 \to T$ **do**
  Train: Huấn luyện $h_t = \arg\min_{h \in \mathcal{H}} \sum_{i=1}^{N} w_t(i) \mathbf{1}[h(\mathbf{x}_i) \neq y_i]$.
  Evaluate: Tính sai số có trọng số $\epsilon_t = \sum_{i=1}^{N} w_t(i) \mathbf{1}[h_t(\mathbf{x}_i) \neq y_i]$.
  if $\epsilon_t = 0$ **then**
    return $h_t(\mathbf{x})$ (dừng sớm)
  end if
  if $\epsilon_t \geq 1 - 1/K$ **then**
    Loại h_t và dừng boosting.
  end if
  Weight: Tính trọng số mô hình $\alpha_t = \eta \left[\ln\left(\frac{1-\epsilon_t}{\epsilon_t}\right) + \ln(K-1) \right]$.
  Update: Cập nhật trọng số mẫu $\widetilde{w}_{t+1}(i) = w_t(i) \exp(\alpha_t \mathbf{1}[h_t(\mathbf{x}_i) \neq y_i])$.
  Chuẩn hóa: $Z_t = \sum_j \widetilde{w}_{t+1}(j)$, $w_{t+1}(i) = \widetilde{w}_{t+1}(i)/Z_t$.
end for
return $H(\mathbf{x}) = \arg\max_{k \in \{0,\ldots,K-1\}} \sum_{t=1}^{T'} \alpha_t \mathbf{1}[h_t(\mathbf{x}) = k]$ (với $T' \leq T$ là số vòng thực tế đã huấn luyện).

6.2 Khảo sát siêu tham số và đánh giá thực nghiệm AdaBoost

6.2.1 Khảo sát Accuracy theo thuật toán phân loại

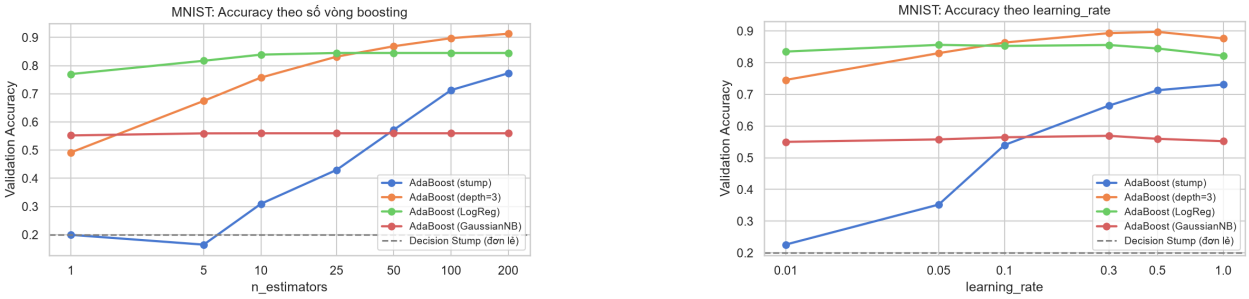
Đối chiếu Accuracy của AdaBoost khi sử dụng Decision Tree (depth = 1), Decision Tree (depth = 3), Logistic Regression và Gaussian Naive Bayes với `learning_rate` = 0,5. Trên Validation, Decision Tree (depth = 3) đạt cao nhất (91,30%), tiếp theo là Logistic Regression (84,48%), Decision Tree (depth = 1) (77,28%) và Gaussian Naive Bayes (55,98%). Kết hợp với khảo sát siêu tham số, chọn depth = 3, `n_estimators` = 200 và `learning_rate` = 0,5 để đánh giá trên tập Test độc lập.



Hình 15: So sánh Accuracy trên Train / Validation / Test.

6.2.2 Khảo sát siêu tham số

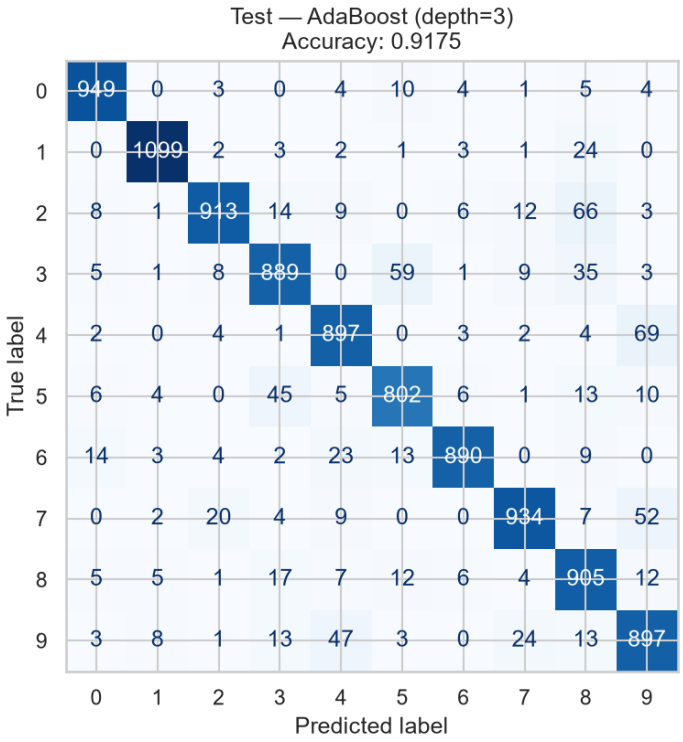
Khảo sát tác động của `n_estimators` và `learning_rate` đến Accuracy trên 10.000 ảnh Validation sau khi huấn luyện bằng 50.000 ảnh Train. Decision Tree (depth = 3) dẫn đầu, đạt 91,30% với 200 vòng boosting; khi cố định 100 vòng, `learning_rate` = 0,5 cho kết quả gần 90%.



Hình 16: Validation Accuracy theo số vòng boosting (trái) và tốc độ học (phải).

6.2.3 Ma trận nhầm lẫn trên tập kiểm thử

Ma trận nhầm lẫn đánh giá AdaBoost với Decision Tree (depth = 3) trên 10.000 ảnh Test độc lập, sau khi chọn tham số bằng Validation. Mô hình đạt Accuracy 91,75% và Macro-F1 91,70%; các dự đoán sai tập trung ở những chữ số có nét viết gần nhau, đáng chú ý là cặp 4 và 9.



Hình 17: Ma trận nhầm lẫn trên tập Test – AdaBoost với Decision Tree (depth = 3).

6.2.4 Nhận xét

- Trong bốn phương pháp được khảo sát, AdaBoost kết hợp Decision Tree (depth = 3) đạt Accuracy cao nhất trên tập Validation (91,30%), vượt Logistic Regression (84,48%) và Decision Tree (depth = 1) (77,28%). Kết quả cho thấy việc tăng độ sâu của cây giúp mô hình học được các ranh giới phân loại phức tạp hơn. Trên tập Test, mô hình đạt Accuracy 91,75% và Macro-F1 91,70%, với chênh lệch Accuracy chỉ 0,45 điểm phần trăm so với Validation, cho thấy khả năng tổng quát hóa tương đối ổn định trên hai tập dữ liệu.
- Khảo sát siêu tham số cho thấy việc tăng `n_estimators` giúp cải thiện Accuracy, đặc biệt từ 50 lên 100 vòng, nhưng mức cải thiện giảm dần khi số vòng boosting tăng cao. Đối với `learning_rate`, giá trị quá nhỏ (như 0,01) khiến mô hình hội tụ chậm, trong khi khoảng 0,3–0,5 cho kết quả tốt nhất. Điểm đánh đổi thực nghiệm là việc tăng số vòng boosting cần được cân nhắc với chi phí huấn luyện tăng tuyến tính.
- Ma trận nhầm lẫn cho thấy phần lớn dự đoán tập trung trên đường chéo chính. Tương tự các mô hình khác, khó khăn với các chữ số có nét viết tương đồng. Điển hình là cặp 4 và 9 có 116 mẫu bị nhầm lẫn qua lại, cho thấy giới hạn của mô hình khi phân biệt những đặc trưng hình học tinh tế của hai chữ số này. Đây là nguyên nhân khiến AdaBoost chưa đạt độ chính xác cao hơn trên tập Test.

7 So sánh tổng hợp và Đánh giá các tiêu chí

7.1 Bảng tổng hợp chỉ số metrics và thời gian chạy

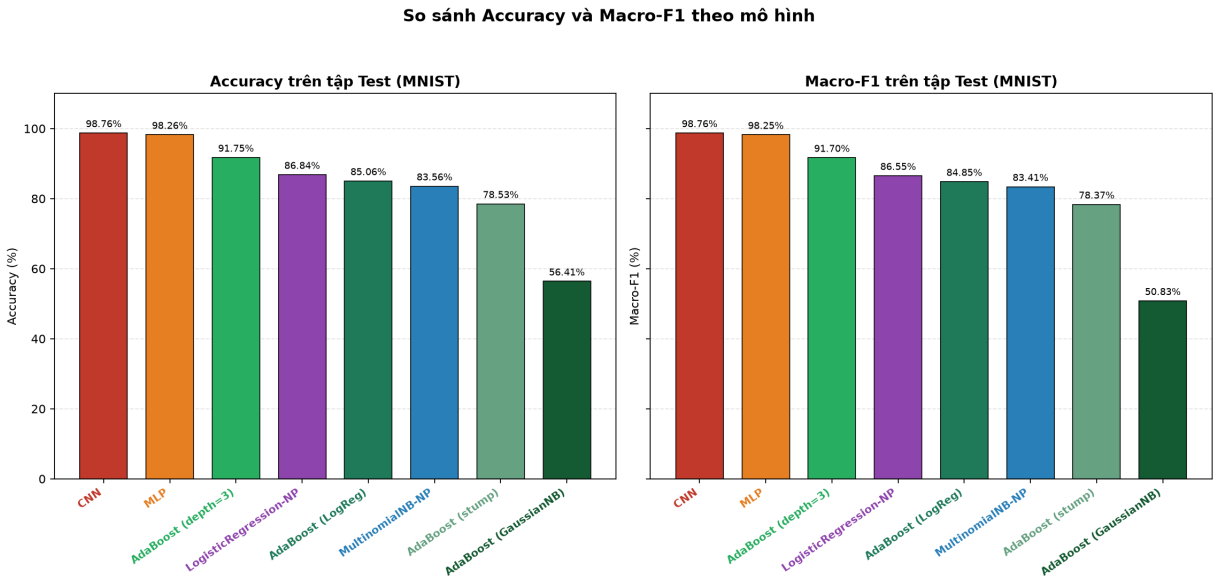
Bảng tổng hợp kết quả đo đạc trên cả 3 tập dữ liệu Train/Validation/Test của toàn bộ các phương pháp trong đề tài.

Bảng 5: Kết quả thực nghiệm của năm mô hình đại diện (Train / Validation / Test).

Mô hình	Tập	Acc	Prec	Rec	F1	Train (s)	Pred (s)
Multinomial Naive Bayes	Train (50000)	82.47	83.19	82.18	82.35	0.0541	1.4082
	Val (10000)	82.15	82.77	81.80	81.98	–	0.2857
	Test (10000)	83.56	84.24	83.25	83.41	–	0.2689
Logistic Regression (OvR)	Train (50000)	86.00	86.04	85.72	85.71	10.7028	0.1679
	Val (10000)	85.96	85.96	85.64	85.69	–	0.0284
	Test (10000)	86.84	86.87	86.58	86.55	–	0.0288
AdaBoost (depth=3)	Train (50000)	92.28	92.28	92.24	92.23	11019.2396	4.9749
	Val (10000)	91.30	91.28	91.22	91.22	–	2.7632
	Test (10000)	91.75	91.81	91.70	91.70	–	2.6681
Multi-Layer Perceptron	Train (50000)	100.00	100.00	100.00	100.00	33.5715	0.1142
	Val (10000)	97.88	97.85	97.87	97.86	–	0.0230
	Test (10000)	98.26	98.25	98.24	98.25	–	0.0216
Convolutional Neural Net	Train (50000)	99.65	99.65	99.65	99.65	51.6119	2.1970
	Val (10000)	98.58	98.57	98.57	98.57	–	0.4391
	Test (10000)	98.76	98.77	98.75	98.76	–	0.4285

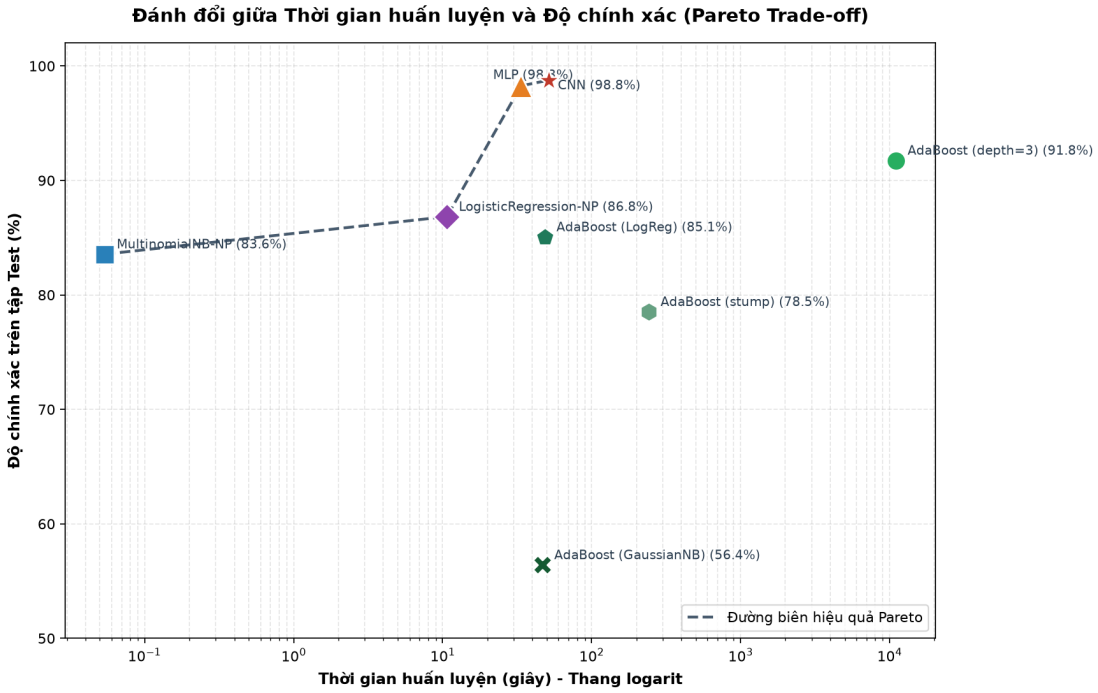
7.2 Diễn giải kết quả và đánh giá cơ bản các mô hình

1. **Naive Bayes (83,56%):** huấn luyện nhanh nhất (0,0541 s) nhưng bị giới hạn bởi giả định độc lập đặc trưng.
2. **Logistic OvR (86,84%):** suy luận nhanh nhất (0,0288 s/10.000 mẫu), cấu hình chuẩn hóa mới vượt Bayes 3,28 điểm % nhờ tối ưu siêu tham số.
3. **AdaBoost depth=3 (91,75%):** cao hơn Logistic OvR 4,91 điểm % và cao hơn Naive Bayes 8,19 điểm %, đổi lại thời gian huấn luyện 11.019,24 s.
4. **MLP (98,26%):** đạt gần CNN và có thời gian dự đoán thấp nhất (0,0216 s/10.000 mẫu).
5. **CNN (98,76%):** tốt nhất về Accuracy và Macro-F1 nhờ khai thác cấu trúc không gian của ảnh.



(a) Accuracy và Macro-F1 của 8 cấu hình.

CNN đạt hiệu năng tốt nhất (Accuracy: 98,77%, Macro-F1: 98,77%), nhỉnh hơn 0,51% so với MLP (98,26% và 98,25%). Trong nhóm học máy truyền thống, AdaBoost (depth=3) cao nhất (91,75%), theo sau là Logistic Regression (86,84%). Các mô hình còn lại như MultinomialNB, AdaBoost (stump/LogReg/GaussianNB) cho kết quả từ 56,41% đến 85,06%.

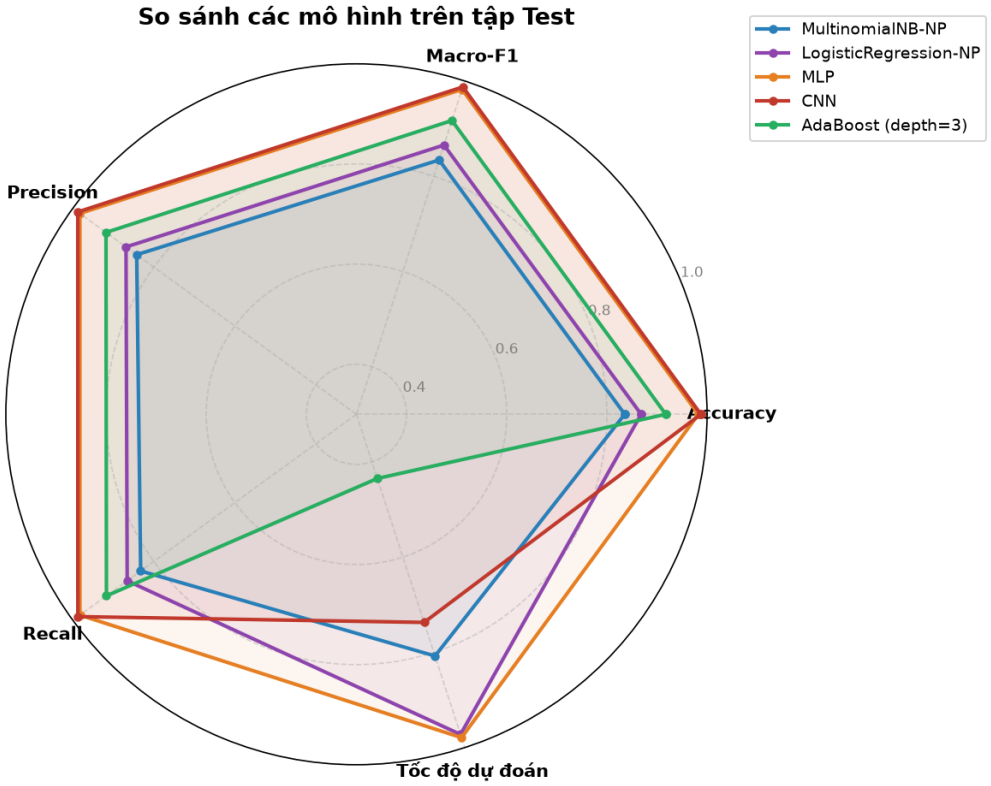


(a) Đánh đổi thời gian huấn luyện – Accuracy.

Hình 13: So sánh định lượng và biên hiệu quả Pareto từ dữ liệu thực nghiệm.

MultinomialNB có tốc độ huấn luyện nhanh nhất (0,06 giây). CNN (43,6 giây) và MLP (30,3 giây) mất nhiều thời gian hơn nhưng đánh đổi bằng độ chính xác vượt trội. Ngược lại, AdaBoost (depth=3) mất tới 3 giờ (11.019 giây) huấn luyện do cấu trúc tuần tự trên ảnh phẳng, nhưng hiệu năng vẫn thấp hơn CNN 7,02% do không tận dụng được đặc trưng không gian của dữ liệu ảnh.

7.3 Phân tích và đưa ra kết luận mô hình tối ưu



Hình 14: Biểu đồ Radar đánh giá đa tiêu chí của năm mô hình.

Việc lựa chọn mô hình tối ưu phụ thuộc vào bài toán thực tế và sự đánh đổi giữa ba yếu tố cốt lõi: độ chính xác, tốc độ suy luận và chi phí huấn luyện. Dựa trên phân tích đa tiêu chí, kết quả thực nghiệm cho thấy các định hướng lựa chọn như sau:

- **Ưu tiên độ chính xác tối đa:** CNN là lựa chọn hàng đầu nhờ khả năng khai thác cấu trúc ảnh 2D, đạt Accuracy 98,76% với sai số chỉ 1,24%. Tuy nhiên, mô hình này đòi hỏi thời gian huấn luyện lâu hơn và độ trễ suy luận (42,85 μs/mẫu) cao hơn so với các mô hình khác.
- **Cân bằng giữa hiệu năng và tốc độ (Lựa chọn tối ưu nhất):** MLP mang lại sự cân bằng xuất sắc. Dù Accuracy (98,26%) thấp hơn CNN 0,50 điểm phần trăm, MLP sở hữu tốc độ suy diễn nhanh nhất (chỉ 2,16 μs/mẫu - nhanh hơn CNN gần 20 lần trên phép đo theo lô) và thời gian huấn luyện tương đối ngắn (33,57 giây).
- **Ưu tiên tốc độ huấn luyện và tài nguyên thấp:** Nhóm mô hình học máy truyền thống thể hiện rõ ưu thế này. MultinomialNB có tốc độ huấn luyện nhanh kỷ lục (0,054 giây) đạt Accuracy 83,56%, cực kỳ phù hợp làm baseline hoặc cho các hệ thống cần cập nhật (train) liên tục. Logistic Regression (Accuracy 86,84%) cũng là một giải pháp tốt với độ trễ thấp (2,88 μs/mẫu), cấu trúc nhỏ gọn (7.850 trọng số) và tính diễn giải cao.
- **Hạn chế của mô hình quần thể:** AdaBoost (depth=3) không khả thi cho bài toán này. Dù đạt độ chính xác khá (91,75%), mô hình tiêu tốn lượng tài nguyên khổng lồ (hơn 3 giờ huấn luyện) và độ trễ suy luận rất cao (266,81 μs/mẫu).
- **Biên hiệu quả Pareto:** Khi xét đồng thời độ chính xác và thời gian huấn luyện, các mô hình MultinomialNB, Logistic Regression, MLP và CNN thiết lập nên một biên Pareto chuẩn mực (không có mô hình nào vượt trội hơn ở cả hai tiêu chí cùng lúc). Ngược lại, toàn bộ các cấu hình AdaBoost đều bị lấn át hoàn toàn, rơi ra khỏi đường biên này.

Kết luận: Việc triển khai mô hình vào thực tế sẽ tùy thuộc vào ràng buộc tài nguyên. CNN là giải pháp toàn diện nhất khi đặt độ chính xác làm trọng tâm. MLP là phương án tối ưu để triển khai ở môi trường cần suy luận (inference) nhanh nhưng vẫn đảm bảo độ chính xác cao. Cuối cùng, MultinomialNB và Logistic Regression đáp ứng hoàn hảo cho các tác vụ cần baseline siêu nhẹ, dễ diễn giải và có khả năng huấn luyện lại tức thì.

8 Kết luận và Hướng phát triển

8.1 Kết luận

Đồ án đã đạt được các kết quả nổi bật sau:

1. Hoàn thiện pipeline tiền xử lý: xử lý 67 đặc trưng pixel có phương sai bằng 0 và ổn định tính toán số học bằng sigmoid phân nhánh, Log-Sum-Exp và Softmax dịch ngưỡng.
2. Hiện thực hóa 5 mô hình từ cơ bản đến phức tạp: cài đặt thuần NumPy cho Multinomial Naive Bayes (83,56%) và OvR Logistic Regression (82,53%); triển khai học quần thể AdaBoost (91,75%), và mạng học sâu MLP (98,26%), CNN (98,76%).
3. Xây dựng hệ thống đánh giá thực nghiệm minh bạch và trực quan thông qua biểu đồ đánh đổi (Pareto), bản đồ nhiệt và ma trận nhầm lẫn.

8.2 Hạn chế và Hướng phát triển

- **Hạn chế:** Mô hình chưa được mở rộng thông qua các kỹ thuật tăng cường dữ liệu (Data Augmentation) như xoay, co giãn ảnh.
- **Hướng phát triển ngắn hạn:** Nghiên cứu phương pháp Ensemble (Soft-Voting) giữa CNN và AdaBoost; thử nghiệm kiến trúc ResNet và kỹ thuật lượng tử hóa (INT8) để triển khai trên vi điều khiển.
- **Hướng phát triển dài hạn - Hệ thống Web Demo Trực quan:** Xây dựng một nền tảng Web App nhận dạng chữ số viết tay toàn diện, cho phép tương tác trực tiếp với các mô hình:
  - **Đa kênh đầu vào:** Cho phép vẽ trực tiếp trên canvas, tải ảnh lên hoặc chọn từ tập MNIST.
  - **Phân tích đối chứng:** Dự đoán và so sánh chéo kết quả, xác suất, độ tin cậy của 5 mô hình trên cùng một mẫu dữ liệu.
  - **Khám phá "Hộp đen" AI:** Trực quan hóa không gian 3D quá trình forward pass, quan sát feature map, kernel CNN và hoạt động của từng neuron từ dữ liệu thực tế.
  - **Huấn luyện tương tác:** Cho phép người dùng tùy chỉnh tham số (epoch, learning rate, số mẫu), huấn luyện lại mô hình trực tiếp trên trình duyệt và xuất báo cáo (JSON trace).

  Trải nghiệm hệ thống Web Demo tại:
  https://pattern-recognition-mnist-handwritten-digit-recognition.r2u-hcmus.workers.dev/

Phân công công việc nhóm và Khai báo sử dụng AI

Thành viên	MSSV	Phần việc chi tiết đảm nhiệm	Đóng góp
Trần Nguyên Hưng	24280048	• Tiền xử lý dữ liệu MNIST và phân tích khám phá (EDA). • Cài đặt thuần (From Scratch): Multinomial Naive Bayes, OvR Logistic Regression. • Tinh chỉnh siêu tham số (α, η), phân tích bản đồ trọng số & ma trận nhầm lẫn. • Chịu trách nhiệm tổng hợp và soạn thảo báo cáo.	33.3%
Vương Thành Đạt	24280058	• Cài đặt và khảo sát mạng nơ-ron đa tầng (MLP). • Khảo sát Inverted Dropout, tiêu biến gradient, khởi tạo trọng số (He/Xavier). • Thiết kế kiến trúc và huấn luyện mạng nơ-ron tích chập (CNN). • Thiết kế slide, chuẩn bị thuyết trình và hiệu chỉnh báo cáo.	33.3%
Phạm Phương Thảo	24280106	• Nghiên cứu và cài đặt học quần thể (AdaBoost Ensemble). • Khảo sát tham số $N_{\text{estimators}}$, trích xuất độ quan trọng (Feature Importances). • Đánh giá, tổng hợp và so sánh toàn diện hiệu năng các mô hình. • Xây dựng và hoàn thiện hệ thống Web Demo trực quan.	33.4%

Khai báo sử dụng công cụ AI:

Trong quá trình thực hiện đồ án, nhóm có sử dụng các trợ lý AI (Gemini/ChatGPT) với vai trò công cụ hỗ trợ tiện ích nhằm tăng hiệu suất làm việc. Cụ thể, AI được dùng để tra cứu các cú pháp (NumPy, Matplotlib),tinh chỉnh định dạng văn bản LaTeX và hỗ trợ viết web deploy chương trình với mã nguồn của nhóm.

Nhóm khẳng định toàn bộ flow thiết kế, mã nguồn hiện thực thuật toán lõi, cũng như quá trình phân tích đều được các thành viên tự nghiên cứu, bám bản chất bài dạy. Nhóm hoàn toàn làm chủ và chịu trách nhiệm về tính học thuật của đồ án.

Mã nguồn và Tài nguyên Thực nghiệm (GitHub Repository)

Toàn bộ mã nguồn cài đặt các thuật toán (From Scratch & thư viện), pipeline tiền xử lý, kịch bản đánh giá cùng mã nguồn Web Demo được công bố mở tại:

https://github.com/gautrangtoidayyy/Pattern-Recognition

Tài liệu tham khảo

[1] Trương Phước Hưng, *Slide Bài giảng Nhận dạng mẫu: Buổi 1-2 (Bayes Decision Theory), Buổi 3 (Linear Classifiers), Buổi 4 (MLP), Buổi 5 (Ensemble)*, Khoa CNTT, ĐH KHTN ĐHQG-HCM, 2026.

[2] Lê Phú Trường, *Source code mẫu: Lab 2, Lab 3, Lab 4, Lab 5.*

[3] Christopher M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006. Mục 4.3.2 (Logistic Regression).

[4] Sergios Theodoridis, Konstantinos Koutroumbas, *Pattern Recognition*, 4th Edition, Academic Press, 2008.

[5] Yann LeCun, Corinna Cortes, Christopher J.C. Burges, *The MNIST Database of Handwritten Digits*, 1998.